MODULE 6

PROGRAMMATION ET AUTONOMIE LOGICIELLE

CHAPITRE 6.7

PLANIFICATION DE TRAJECTOIRE ET ÉVITEMENT D'OBSTACLES

Niveau : *intermédiaire.*

Prérequis : *chapitre 6.5, estimation d'état ; chapitre 6.6, navigation visuelle et localisation sans GPS ; chapitre 6.4, envoi de consignes depuis le calculateur compagnon ; chapitre 3.5, capteurs de perception et de profondeur.*

Régime de profondeur : *choix. Le chapitre outille l'étudiant pour arbitrer entre planification globale et évitement local, et entre les algorithmes de chaque famille, selon la connaissance de l'environnement et les contraintes de la mission, jusqu'aux critères de décision et à leurs limites.*

Notions de programmation introduites : *la planification globale sur graphe ; la planification par échantillonnage ; l'évitement local réactif et les champs de potentiels ; les notions de complétude et d'optimalité ; la distance d'arrêt et la marge de sécurité.*

Paramètres ArduPilot introduits : `OA_TYPE`, *qui choisit l'algorithme d'évitement d'obstacles, et* `OA_MARGIN_MAX`, *qui fixe la marge de sécurité à maintenir autour des obstacles.*

Budget de temps : *lecture environ 1 h ; reprise de l'exemple résolu environ 30 min ; travail dirigé environ 1 h ; auto-évaluation environ 15 min.*

Objectifs : *à la fin de ce chapitre, vous saurez distinguer la planification globale de l'évitement local et expliquer leur complémentarité ; décrire Dijkstra, A*, RRT et les champs de potentiels avec leurs propriétés ; calculer une distance d'arrêt et la portée de détection nécessaire à un évitement sûr, et en déduire une vitesse maximale sûre ; et configurer l'évitement d'ArduPilot par* `OA_TYPE` *et* `OA_MARGIN_MAX`.

Livrables : *un calcul de distance d'arrêt et de vitesse maximale sûre ; une note comparant planification globale et évitement local ; une configuration d'évitement.*

6.7.1. Décider où aller

Les chapitres précédents ont donné à l'appareil la connaissance de sa position, avec ou sans GPS. Reste à décider où aller. Ce problème se décompose en deux, de portées différentes. Le premier est la planification globale : sur une carte connue à l'avance, trouver un chemin du point de départ au but qui contourne les obstacles répertoriés. Le second est l'évitement local : en vol, réagir à un obstacle imprévu que les capteurs viennent de détecter, et que la carte ne connaissait pas.

Une image aide à saisir la différence. Un GPS routier calcule à l'avance l'itinéraire complet vers une destination, c'est la planification globale ; le conducteur qui freine devant un piéton surgi n'attend pas que le GPS recalcule, il réagit, c'est l'évitement local. Le premier a la vue d'ensemble et le temps de réfléchir, le second n'a que l'instant et ce qu'il voit devant lui. Un drone autonome combine les deux exactement pour les mêmes raisons, et cette analogie routière guide tout le chapitre.

Ces deux problèmes sont complémentaires et ne s'opposent pas. La planification globale trace la route d'ensemble, mais elle ignore ce qui n'était pas sur la carte, une grue installée depuis, un câble, une personne. L'évitement local rattrape ces surprises, mais il n'a pas de vue d'ensemble et peut s'engager dans une impasse qu'un planificateur global aurait évitée. Un système complet emploie les deux, la planification pour la stratégie et l'évitement pour

la réaction, et c'est d'ailleurs ce que fait l'évitement d'ArduPilot. Les deux familles reposent sur des algorithmes distincts, que les sections suivantes présentent.

6.7.2. La planification globale, trouver un chemin sur une carte

La planification globale suppose une carte, souvent représentée par un graphe ou une grille où chaque case est libre ou occupée. Trouver un chemin revient alors à trouver une suite de cases libres reliant le départ au but. L'algorithme de Dijkstra résout ce problème en explorant les cases par ordre de distance croissante depuis le départ, et garantit le plus court chemin [F20]. Sa faiblesse est d'explorer dans toutes les directions, y compris à l'opposé du but, ce qui coûte du calcul inutile.

L'algorithme A* corrige ce défaut par une heuristique, une estimation de la distance restante jusqu'au but, qui oriente l'exploration vers le but plutôt qu'en aveugle [F21]. À condition que cette heuristique ne surestime jamais la distance réelle, A* trouve lui aussi le plus court chemin, mais bien plus vite que Dijkstra. C'est le planificateur de référence sur grille, et l'un des deux que met en œuvre ArduPilot.

Ces méthodes sur grille conviennent à un espace en deux dimensions bien discrétisé. Pour des espaces de plus grande dimension ou continus, une autre famille, la planification par échantillonnage, tire au hasard des points de l'espace et les relie en un arbre qui croît vers les zones inexplorées, méthode dite RRT [F22]. Sa variante RRT* améliore peu à peu le chemin trouvé vers l'optimum à mesure que l'échantillonnage se poursuit [F23]. Ces approches trouvent une solution quand elle existe, sans garantir la plus courte à moins de laisser tourner l'algorithme. L'ouvrage de référence sur ces méthodes en donne le panorama complet [F19].

Le choix entre ces familles se raisonne. Sur une grille en deux dimensions bien connue, A* est le plus efficace et donne le chemin optimal. En trois dimensions, ou lorsqu'il faut tenir compte des contraintes de mouvement de l'appareil, qui ne peut pas tourner sur place ni s'arrêter net, l'échantillonnage de RRT passe mieux à l'échelle. La finesse de la grille est elle-même un compromis : trop grossière, elle rate des passages étroits ou coupe des angles ; trop fine, elle fait exploser le nombre de cases à explorer et le temps de calcul.

6.7.3. L'évitement local, réagir aux obstacles perçus

L'évitement local ne dispose pas d'une carte, mais de ce que les capteurs perçoivent à l'instant, un télémètre de proximité, une caméra de profondeur du chapitre 3.5. Il doit décider vite d'une manœuvre pour ne pas heurter un obstacle qui vient d'apparaître. Une méthode classique est celle des champs de potentiels [F24]. On imagine le but comme un puits qui attire l'appareil, et chaque obstacle comme une colline qui le repousse. La somme

de ces forces, attractive vers le but et répulsive depuis les obstacles, donne à chaque instant la direction à suivre.

Cette méthode est simple et rapide, mais elle a un défaut connu, les minima locaux. Dans une configuration où l'attraction du but et la répulsion d'un obstacle s'annulent, par exemple face à un obstacle concave ou dans un couloir en impasse, la force résultante devient nulle et l'appareil se fige, incapable d'avancer alors qu'un chemin existe. Ce piège est la limite structurelle des méthodes purement réactives, qui n'ont pas la vue d'ensemble d'un planificateur.

L'évitement de BendyRuler, employé par ArduPilot, procède autrement. Plutôt que de sommer des forces, il sonde plusieurs directions autour de celle du but, sur une certaine distance devant lui, et choisit la direction dégagée la plus proche de celle du but. Ce balayage local évite mieux les petits obstacles qu'un champ de potentiels, mais il reste réactif et peut, lui aussi, s'engager dans une impasse qu'une planification globale aurait vue.

Ces méthodes réactives dépendent entièrement de la qualité de la perception. Un obstacle non détecté, parce que trop fin comme un câble, trop sombre, ou hors du champ du capteur, ne sera pas évité, quel que soit l'algorithme. C'est pourquoi l'évitement se pense toujours avec le capteur qui l'alimente, et non comme une brique logicielle isolée. Le chapitre 3.5 a présenté ces capteurs ; leur portée et leur champ de vue conditionnent directement ce que l'évitement peut faire.

6.7.4. La géométrie de l'évitement, distance d'arrêt et marge

Un évitement, aussi intelligent soit-il, se heurte à une contrainte physique : l'appareil ne s'arrête pas instantanément. Entre l'instant où l'obstacle est détecté et l'instant où l'appareil est stoppé ou dévié, il parcourt une distance qui dépend de sa vitesse. Cette distance d'arrêt comprend deux termes. Le premier est la distance de réaction, parcourue pendant la latence de perception et de décision étudiée aux chapitres 6.3 et 6.4, proportionnelle à la vitesse. Le second est la distance de freinage, parcourue pendant le ralentissement, qui croît comme le carré de la vitesse.

La conséquence est que la portée de détection nécessaire n'augmente pas proportionnellement à la vitesse, mais bien plus vite, à cause du terme de freinage. Doubler la vitesse quadruple à peu près la distance de freinage. Un capteur de portée donnée impose donc une vitesse maximale au-delà de laquelle l'appareil ne peut plus s'arrêter à temps, quelle que soit la qualité de l'algorithme d'évitement. À cette distance d'arrêt s'ajoute une marge de sécurité, qui garde une distance de confort avec l'obstacle et absorbe les incertitudes de mesure. L'exemple résolu chiffre ce budget et en tire la vitesse maximale sûre.

Ce raisonnement a une portée pratique immédiate. Il permet, connaissant la portée du capteur embarqué, de fixer une vitesse de croisière qui laisse toujours de quoi s'arrêter, plutôt que de découvrir la limite lors d'une collision. Il explique aussi pourquoi les drones qui volent vite en évitement, dans une course ou une inspection rapide, exigent des capteurs à longue portée et une chaîne de perception à faible latence, les deux termes de la distance d'arrêt étant attaqués de front. La sécurité de l'évitement se dimensionne donc au moment de choisir le capteur et la vitesse de mission, bien avant d'écrire la moindre ligne d'évitement.

6.7.5. L'évitement dans ArduPilot

ArduPilot réunit les deux familles dans sa fonction d'évitement d'obstacles [F08]. Le paramètre `OA_TYPE` choisit l'algorithme : la valeur BendyRuler active l'évitement local réactif, la valeur Dijkstra active la planification globale autour des zones interdites, et une troisième valeur combine les deux, Dijkstra pour la route d'ensemble et BendyRuler pour les surprises. Ce choix reflète directement la complémentarité de la section 6.7.1, matérialisée en un paramètre.

Le paramètre `OA_MARGIN_MAX` fixe la marge de sécurité, la distance que l'appareil cherche à garder des obstacles. Une marge trop faible frôle les obstacles et laisse peu de place à l'erreur ; une marge trop grande refuse des passages pourtant franchissables et fige l'appareil dans un environnement encombré. Sa valeur se choisit selon la précision de la perception et l'encombrement de l'espace. L'évitement s'appuie sur les capteurs de proximité du chapitre 3.5 et agit dans les modes AUTO, GUIDED et RTL. En simulation, on définit une zone interdite ou un obstacle, on règle `OA_TYPE` et `OA_MARGIN_MAX`, et l'on observe la trajectoire se courber pour contourner l'obstacle en gardant la marge demandée.

L'évitement s'appuie sur deux entrées distinctes. Les clôtures, définies à l'avance, décrivent les zones interdites connues, que Dijkstra contourne. Les capteurs de proximité, télémètres ou caméra de profondeur, signalent les obstacles imprévus, que BendyRuler évite. La combinaison des deux valeurs de `OA_TYPE` traite ainsi le connu et l'imprévu dans une même trajectoire, ce qui est le comportement le plus complet et le plus prudent. Régler l'un sans l'autre laisse un angle mort, une carte sans capteur ignorant les surprises, un capteur sans carte s'engageant dans des impasses.

6.7.6. Exemple résolu

Énoncé. Un drone équipé d'une caméra de profondeur de 10 m de portée doit éviter des obstacles imprévus. On veut calculer sa distance d'arrêt à 5 m/s, la portée de détection que cela exige avec une marge de 2 m, et la vitesse maximale à laquelle il peut encore s'arrêter à temps avec ce capteur.

Données et hypothèses.

Latence de perception et de décision, $t_r = 0,2$ s, ordre de grandeur des chaînes des chapitres 6.3 et 6.4.

Décélération de freinage, $a = 3$ m/s², valeur prudente pour un multirotor qui doit rester stable.

Marge de sécurité, OA_MARGIN_MAX prise à 2 m. Portée du capteur, 10 m.

Modèle de distance d'arrêt à décélération constante, valable tant que le freinage reste dans le domaine de vol stable de l'appareil. Une décélération trop forte sortirait de ce domaine et invaliderait le modèle.

Étape 1, distance d'arrêt à 5 m/s. La distance d'arrêt est la somme d'une distance de réaction et d'une distance de freinage :

$$d_{arret} = v\,t_r + \frac{v^2}{2a}$$

où d_{arret} est en mètres, v la vitesse en mètres par seconde, t_r la latence en secondes et a la décélération en mètres par seconde carré. Le premier terme croît avec la vitesse, le second avec son carré. Pour 5 m/s :

$$d_{arret} = 5 \times 0,2 + \frac{5^2}{2\times3} = 1,00 + 4,17 = 5,17\ m$$

Étape 2, portée de détection nécessaire. En ajoutant la marge de sécurité :

$$d_{detection} = d_{arret} + marge = 5,17 + 2 = 7,17\ m$$

La caméra de 10 m suffit donc à 5 m/s, avec une réserve confortable de près de 3 m.

Étape 3, vitesse maximale sûre. La portée disponible pour l'arrêt est celle du capteur moins la marge, soit 8 m. On cherche la vitesse telle que la distance d'arrêt égale 8 m, en résolvant l'équation du second degré :

$$\frac{v^2}{2a} + t_r v - 8 = 0 \quad \Rightarrow \quad v_{max} = 6,35\ m/s$$

Au-delà de 6,35 m/s, la distance d'arrêt dépasse la portée utile du capteur, et l'appareil ne peut plus s'arrêter à temps face à un obstacle vu à la limite de sa portée.

Étape 4, analyse de sensibilité. Le tableau ci-dessous montre la croissance rapide de la distance d'arrêt avec la vitesse. Entre 5 et 10 m/s, la distance d'arrêt passe de 5,17 à 18,67 m, plus que triplée pour une vitesse seulement doublée, à cause du terme en carré. Le résultat correct s'énonce ainsi : la portée du capteur impose une vitesse maximale de vol sûr, et vouloir voler plus vite exige un capteur à plus longue portée, non un meilleur algorithme. La marge OA_MARGIN_MAX déplace ce seuil sans changer sa nature. La sécurité de l'évitement est d'abord une affaire de physique et de portée, avant d'être une affaire de logiciel.

vitesse	distance d'arrêt	portée requise
2 m/s	1,07 m	3,07 m
5 m/s	5,17 m	7,17 m
8 m/s	12,27 m	14,27 m
10 m/s	18,67 m	20,67 m

```python
# Distance d'arret et portee de detection pour un evitement sur. Python 3.11, validite 2026.
def distance_arret(v, t_r, a):
    return v * t_r + v**2 / (2 * a)   # reaction + freinage

t_r, a, marge = 0.2, 3.0, 2.0
for v in (2, 5, 10):
    d = distance_arret(v, t_r, a)
    print(f"v = {v:2d} m/s -> arret {d:5.2f} m, portee requise {d + marge:5.2f} m")
```

Sortie attendue :

```
v =  2 m/s -> arret  1.07 m, portee requise  3.07 m
v =  5 m/s -> arret  5.17 m, portee requise  7.17 m
v = 10 m/s -> arret 18.67 m, portee requise 20.67 m
```

6.7.7. Pièges et erreurs fréquentes

Croire qu'un meilleur algorithme d'évitement lève la contrainte de portée est l'erreur que l'exemple résolu corrige. Aucun algorithme ne fait s'arrêter l'appareil plus court que la physique ne le permet. La portée du capteur et la vitesse fixent une limite que le logiciel ne franchit pas.

Compter sur un évitement local seul dans un environnement piégeux expose aux minima locaux. Un champ de potentiels ou un balayage réactif peut figer l'appareil face à un obstacle concave ou l'engager dans une impasse. Une planification globale est nécessaire dès que l'environnement présente ce genre de pièges.

Régler `OA_MARGIN_MAX` sans tenir compte de l'encombrement conduit à deux échecs symétriques. Une marge trop faible frôle les obstacles, une marge trop grande refuse des passages franchissables et bloque l'appareil. La marge se règle selon la précision de perception et l'espace disponible.

Employer une heuristique qui surestime la distance restante casse la garantie d'optimalité de A*. Si l'heuristique dépasse la distance réelle, A* peut retourner un chemin qui n'est pas le plus court. L'heuristique doit toujours minorer la distance réelle.

6.7.8. Travail dirigé

Objectif. Établir le budget de portée d'un évitement et configurer l'évitement d'ArduPilot en simulation.

Environnement. Python 3.11 ; ArduPilot SITL et Mission Planner suivant le chapitre 2.12 ; le guide d'évitement d'obstacles d'ArduPilot. Aucun matériel embarqué.

Durée. 1 h.

Procédure.

1. Reprendre la fonction de distance d'arrêt de l'exemple résolu et tracer la portée requise en fonction de la vitesse, de 2 à 15 m/s.
2. Pour une portée de capteur de 6 m, puis de 12 m, calculer la vitesse maximale sûre avec une marge de 2 m, et comparer.
3. Dans SITL, définir une clôture d'exclusion ou un obstacle, régler `OA_TYPE` sur BendyRuler puis sur la combinaison Dijkstra et BendyRuler, et observer la différence de trajectoire.
4. Faire varier `OA_MARGIN_MAX` et constater son effet sur la distance de contournement.

Résultat attendu. Une courbe de portée requise, deux vitesses maximales sûres calculées, et deux captures de trajectoire montrant l'effet du type d'évitement et de la marge.

Critères d'auto-validation. À 5 m/s, la portée requise doit valoir 7,17 m. Pour un capteur de 6 m, la vitesse maximale sûre avec 2 m de marge doit être inférieure à celle d'un capteur de 12 m, ce qui confirme que la portée fixe la vitesse. En simulation, la combinaison Dijkstra et BendyRuler doit contourner plus intelligemment un obstacle concave que BendyRuler seul, qui peut s'y attarder. Une marge plus grande doit éloigner la trajectoire de l'obstacle.

6.7.9. Auto-évaluation

Question 1. Quelle est la différence de portée entre planification globale et évitement local ?

A. La planification globale trace la route sur une carte connue, l'évitement local réagit aux obstacles imprévus.
B. La planification globale est toujours plus rapide, l'évitement local toujours plus précis.
C. Il n'y a aucune différence, les deux termes désignent le même algorithme sous deux noms.
D. La planification globale s'exécute au sol, l'évitement local uniquement dans la station de contrôle.

Question 2. Qu'apporte l'heuristique de l'algorithme A* par rapport à Dijkstra ?

A. Elle garantit une solution même quand aucun chemin n'existe entre le départ et le but.
B. Elle oriente l'exploration vers le but et accélère la recherche du plus court chemin.
C. Elle supprime le besoin de carte, A* explorant l'environnement au hasard.
D. Elle remplace la grille par un arbre d'échantillons tirés dans tout l'espace.

Question 3. Quel est le défaut structurel des méthodes d'évitement purement réactives comme les champs de potentiels ?

A. Elles consomment trop de mémoire, car elles stockent toute la carte de l'environnement.
B. Elles ne fonctionnent qu'avec un GPS, et échouent dès que le signal disparaît.
C. Elles peuvent figer l'appareil dans un minimum local, face à un obstacle concave ou une impasse.
D. Elles trouvent toujours le plus court chemin, mais trop lentement pour le temps réel.

Question 4. Un drone vole à 5 m/s avec 0,2 s de latence et 3 m/s² de freinage. Quelle est sa distance d'arrêt, et que se passe-t-il si la vitesse double ?

A. Environ 5,2 m, et elle double simplement à 10,4 m quand la vitesse double.
B. Environ 1 m, car seule la distance de réaction compte dans un arrêt d'urgence.
C. Environ 5,2 m, et elle plus que triple quand la vitesse double, à cause du terme de freinage.
D. Elle est nulle, car un multirotor peut s'arrêter instantanément en inversant sa poussée.

Question 5. Quel paramètre choisit l'algorithme d'évitement d'obstacles d'ArduPilot, et quelles valeurs propose-t-il ?

A. `OA_MARGIN_MAX`, qui bascule entre évitement activé et désactivé selon sa valeur.
B. `EK3_SRC1_POSXY`, qui choisit entre évitement global et évitement local.
C. `FS_GCS_ENABLE`, qui déclenche l'évitement quand la liaison est perdue.
D. `OA_TYPE`, qui propose BendyRuler, Dijkstra, ou la combinaison des deux.

Les corrigés de cette auto-évaluation se trouvent dans le fichier « Corrigés des auto-évaluations, Module 6 ».

6.7.10. À retenir

Décider où aller se décompose en deux problèmes complémentaires : la planification globale, qui trace une route sur une carte connue, et l'évitement local, qui réagit aux obstacles imprévus perçus en vol. Un système complet emploie les deux.

La planification globale sur graphe repose sur Dijkstra, qui garantit le plus court chemin en explorant en aveugle, et sur A*, qui accélère la recherche par une heuristique minorant la distance restante. La planification par échantillonnage, RRT et RRT*, traite les espaces de grande dimension.

L'évitement local réactif, champs de potentiels ou BendyRuler, décide vite mais souffre des minima locaux, où l'appareil se fige face à un obstacle concave ou une impasse. Seule une vue d'ensemble globale évite ce piège.

La distance d'arrêt vaut $d = v\,t_r + v^2/(2a)$. Le terme de freinage en carré fait croître vite la portée de détection nécessaire, si bien qu'un capteur de portée donnée impose une vitesse maximale de vol sûr, indépendamment de l'algorithme.

ArduPilot réunit les deux familles avec `OA_TYPE`, qui choisit BendyRuler, Dijkstra ou leur combinaison, et `OA_MARGIN_MAX`, qui fixe la marge de sécurité aux obstacles.

6.7.11. Transition

L'appareil sait décider d'une route et d'un évitement. Le chapitre 6.8 passe à l'exécution de cette route, avec le suivi de trajectoire, comment tenir un chemin planifié, et le vol sans GPS, qui rassemble l'estimation, la localisation et la commande des chapitres précédents.

6.7.12. Sources et pour aller plus loin

Les sources citées sont rappelées par leur identifiant de bloc dans la bibliographie consolidée : documentation ArduPilot [D19] ; évitement d'obstacles d'ArduPilot [F08] ; ouvrage de référence sur la planification [F19] ; algorithme de Dijkstra [F20] ; algorithme A* [F21] ; RRT [F22] ; RRT* [F23] ; champs de potentiels [F24].

Pour aller plus loin, l'ouvrage de LaValle sur les algorithmes de planification [F19], en accès libre, développe la complétude et l'optimalité des méthodes présentées et montre comment RRT s'étend aux contraintes de mouvement d'un véhicule réel. Le parcourir après ce chapitre relie les algorithmes vus ici aux contraintes dynamiques que le chapitre 6.8 abordera.